

Aircraft Landing

Problem 1

Sets

- $i \in I = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$: Aircraft

Parameters

- $Target_i$: The target arrival time of aircraft i
- $Late_i$: The latest arrival time of aircraft i
- $Early_i$: The earliest arrival time of aircraft i
- $Sep_{i,j}$: The separation time between aircraft i and j
- δ_i : penalty per minute incurred by aircraft i for landing early
- ρ_i : penalty per minute incurred by aircraft i for landing late

Decision variables

- $t_i \geq 0$: landing time of aircraft i
- $l_i \geq 0$: minutes aircraft i is late
- $e_i \geq 0$: minutes aircraft i is early
- $x_{i,j} \in \{0, 1\}$: whether aircraft i lands before aircraft j (=1) or not (=0)

Model

Objective:

- Minimize the total penalty incurred due to deviations from target landing times :

$$\sum_{i \in I} \delta_i e_i + \rho_i l_i$$

Constraints:

- For each aircraft, connect the landing time with the target time, minutes late, and minutes early:

$$t_i = Target_i - e_i + l_i \quad \forall i$$

- For each aircraft, enforce an upper bound on the amount of delay possible (have to land at the latest at the late arrival time):

$$l_i \leq Late_i - Target_i \quad \forall i$$

- For each aircraft, enforce an upper bound on how many minutes early an aircraft can arrive:

$$e_i \leq Target_i - Early_i \quad \forall i$$

- For all pairs of aircraft (i, j) , with $i \neq j$, enforce the minimum separation time required

$$t_i + Sep_{i,j} \leq t_j + M(1 - x_{i,j}) \quad \forall i, j, i \neq j$$

- For all pairs of aircraft (i, j) , with $i \neq j$, ensure that one of the sequence variables is set

$$x_{i,j} + x_{j,i} = 1 \quad \forall i, j, i \neq j$$

Note that the model can be formulated with only half the number of sequencing variables since a value of $x_{i,j} = 0$ implies aircraft j lands before aircraft i

The full model in Julia/JuMP, is given below:

```
*****
# Aircraft Landing, "Mathematical Programming Modelling" (42112)
# Intro definitions
*****

using JuMP
using HiGHS

Early   = [125 187  94  99 111 123 129 131 142 155]
Target  = [155 212 134 127 123 138 145 157 162 179]
Late    = [473 612 508 421 511 515 525 407 500 612]

delta   = [10 20 35 25 35 30 30 50 30 35] #earliness penalty
rho     = [10 20 35 25 35 30 30 50 30 35] #lateness penalty
```

```

Sep = [0  5  15 15 15 15 15 10 15  5;
       5  0  15 15 10 15 10 15 15 15;
       15 15 0  10 15 10 15 10 15 10;
       15 15 10 0  10 10 10 15 15 15;
       15 10 15 10 0  10 5  10 15 5;
       15 15 10 10 10 0  15 15 15 15;
       15 10 15 10 5  15 0  10 5  10;
       10 15 10 15 10 15 10 0  10 15
       15 15 15 15 15 15 5  10 0  10
       5  15 10 15 5  15 10 15 10 0];

n=10
M=maximum(Late)-minimum(Early)

*****
# MODEL
AL = Model(HiGHS.Optimizer)

#arrival time variables
@variable(AL, t[i=1:n]>=0, Int)
#the number of minutes plane i arrives early
@variable(AL, e[i=1:n]>=0, Int)
#the number of minutes plane i arrives late
@variable(AL, l[i=1:n]>=0, Int)
#binary sequence variables
@variable(AL, x[i=1:n,j=1:n], Bin)

#objective - minimize total penalty
@objective(AL, Min, sum(delta[i]*e[i]+rho[i]*l[i] for i=1:n))

#connection between target time, mins late, mins early, and arrival time
@constraint(AL, [i=1:n], t[i]==Target[i]-e[i]+l[i])
#bounding the mins late and mins early
@constraint(AL, [i=1:n], l[i]<=Late[i]-Target[i])
@constraint(AL, [i=1:n], e[i]<=Target[i]-Early[i])
# enforcing minimum separation time between pairs of aircraft
@constraint(AL, [i=1:n,j=1:n; i!=j], t[i]+Sep[i,j]<=t[j]+(Sep[i,j]+M)*(1-x[i,j]))
# either i before j, or j before i:
@constraint(AL, [i=1:n,j=1:n; i!=j], x[i,j]+x[j,i]==1)

optimize!(AL)
if termination_status(AL) == MOI.OPTIMAL
    println("\nOptimal Objective Value: $(objective_value(AL))")
    println("\nAircraft Landing times\n")
    for i=1:n
        println("Aircraft $i: $(round(value(t[i]),digits=2))")
    end
end

```

```

end
else
println("No Solution:")
end

```

Problem 2

New Variables

- $p_i \geq 0$: The position of aircraft i in the landing sequence

New Constraints

- Convert sequence variables (before/after each other) to position variables for aircraft i :

$$p_i = \sum_{j \in I | j \neq i}^n x_{j,i} + 1 \quad \forall i$$

- Enforce non-consecutive requirement for aircraft 5 and 6

$$p_6 \geq p_5 + 2 - M(1 - x_{5,6})$$

$$p_5 \geq p_6 + 2 - M(1 - x_{6,5})$$

Note how the last two constraints are constructed: If aircraft 6 is after aircraft 5 in the landing sequence it must have a position which is at least two more than aircraft 5, if not the first constraint becomes redundant and the other takes effect instead.

The full model in Julia/JuMP is given below:

```

*****
# Aircraft Landing 2, "Mathematical Programming Modelling" (42112)
# Intro definitions
*****

using JuMP
using HiGHS

Early   = [125 187  94  99 111 123 129 131 142 155]
Target  = [155 212 134 127 123 138 145 157 162 179]
Late    = [473 612 508 421 511 515 525 407 500 612]

delta   = [10 20 35 25 35 30 30 50 30 35] #earliness penalty

```

```

rho      = [10 20 35 25 35 30 30 50 30 35] #lateness penalty

Sep = [0  5  15 15 15 15 15 10 15  5;
       5  0  15 15 10 15 10 15 15 15;
       15 15 0  10 15 10 15 10 15 10;
       15 15 10 0  10 10 10 15 15 15;
       15 10 15 10 0  10 5  10 15 5;
       15 15 10 10 10 0  15 15 15 15;
       15 10 15 10 5  15 0  10 5  10;
       10 15 10 15 10 15 10 0  10 15
       15 15 15 15 15 15 5  10 0  10
       5  15 10 15 5  15 10 15 10 0];

n=10
M=maximum(Late)-minimum(Early)

*****
# MODEL
AL = Model(HiGHS.Optimizer)

#arrival time variables
@variable(AL, t[i=1:n]>=0, Int)
#the number of minutes plane i arrives early
@variable(AL, e[i=1:n]>=0, Int)
#the number of minutes plane i arrives late
@variable(AL, l[i=1:n]>=0, Int)
#binary sequence variables
@variable(AL, x[i=1:n,j=1:n], Bin)
#the position of aircraft i in the landing sequence
@variable(AL, p[i=1:n]>=0)

#objective - minimize total penalty
@objective(AL, Min, sum(delta[i]*e[i]+rho[i]*l[i] for i=1:n))

#onnection between target time, mins late, mins early, and arrival time
@constraint(AL, [i=1:n], t[i]==Target[i]-e[i]+l[i])
#bounding the mins late and mins early
@constraint(AL, [i=1:n], l[i]<=Late[i]-Target[i])
@constraint(AL, [i=1:n], e[i]<=Target[i]-Early[i])
#enforcing minimum separation time between pairs of aircraft
@constraint(AL, [i=1:n,j=1:n; i!=j], t[i]+Sep[i,j]<=t[j]+(Sep[i,j]+M)*(1-x[i,j]))
# either i before j, or j before i:
@constraint(AL, [i=1:n,j=1:n; i!=j], x[i,j]+x[j,i]==1)
#set position variables based on relation variables:
@constraint(AL, [i=1:10], p[i]==sum(x[j,i] for j=1:n if i!=j)+1)
#5/6 not consecutive:

```

```

@constraint(AL, p[6] >= p[5]+2-(n+1)*(1-x[5,6]))
@constraint(AL, p[5] >= p[6]+2-(n+1)*(1-x[6,5]))

optimize!(AL)

if termination_status(AL) == MOI.OPTIMAL
    println("\nOptimal Objective Value: $(objective_value(AL))")
    println("\nAircraft Landing times and position\n")
    for i=1:n
        println("Aircraft $i: time = $(round(value(t[i]),digits=2))
            ), position = $(round(Int,value(p[i])))")
    end
else
    println("No Solution:")
end

```

Problem 3

- To find the second-best sequence we forbid the optimal sequence found previously.

Constraint

- If we denote by x^* , the values of the x variables in the optimal solution, the following constraint can be used to eliminate the optimal landing sequence. In any solution, exactly 45 of the sequencing variables will be set. To forbid the optimal landing sequence we can therefore simply require that at most 44 can be used (implying at least one sequence change)

$$\sum_{(i,j)|x_{i,j}^*=1} x_{i,j} \leq 44$$

The full model in Julia/JuMP is given below:

```

#####
# Aircraft Landing 3, "Mathematical Programming Modelling" (42112)
# Intro definitions
#####

using JuMP
using HiGHS

```

```

Early   = [125 187  94  99 111 123 129 131 142 155]
Target  = [155 212 134 127 123 138 145 157 162 179]
Late    = [473 612 508 421 511 515 525 407 500 612]

delta   = [10 20 35 25 35 30 30 50 30 35] #earliness penalty
rho      = [10 20 35 25 35 30 30 50 30 35] #lateness penalty

Sep = [0  5  15 15 15 15 15 10 15  5;
       5  0  15 15 10 15 10 15 15 15;
       15 15 0  10 15 10 15 10 15 10;
       15 15 10 0  10 10 10 15 15 15;
       15 10 15 10 0  10 5  10 15 5;
       15 15 10 10 10 0  15 15 15 15;
       15 10 15 10 5  15 0  10 5 10;
       10 15 10 15 10 15 10 0  10 15
       15 15 15 15 15 15 5  10 0 10
       5  15 10 15 5  15 10 15 10 0];

n=10
M=maximum(Late)-minimum(Early)

*****
# MODEL (initially same as problem 2)
AL = Model(HiGHS.Optimizer)

#arrival time variables
@variable(AL, t[i=1:n]>=0, Int)
#the number of minutes plane i arrives early
@variable(AL, e[i=1:n]>=0, Int)
#the number of minutes plane i arrives late
@variable(AL, l[i=1:n]>=0, Int)
#binary sequence variables
@variable(AL, x[i=1:n,j=1:n], Bin)
#the position of aircraft i in the landing sequence
@variable(AL, p[i=1:n]>=0)

#objective - minimize total penalty
@objective(AL, Min, sum(delta[i]*e[i]+rho[i]*l[i] for i=1:n))

#onnection between target time, mins late, mins early, and arrival time
@constraint(AL, [i=1:n], t[i]==Target[i]-e[i]+l[i])
#bounding the mins late and mins early
@constraint(AL, [i=1:n], l[i]<=Late[i]-Target[i])
@constraint(AL, [i=1:n], e[i]<=Target[i]-Early[i])
#enforcing minimum separation time between pairs of aircraft
@constraint(AL, [i=1:n,j=1:n; i!=j], t[i]+Sep[i,j]<=t[j]+(Sep[i,j]+M)*(1-x[i,j]))

```

```

@constraint(AL, [i=1:n,j=1:n; i!=j], x[i,j]+x[j,i]==1)
#set position variables
@constraint(AL, [i=1:10], p[i]==sum(x[j,i] for j=1:n if i!=j)+1)
#5/6 not consecutive
@constraint(AL, p[6] >= p[5]+2-(n+1)*(1-x[5,6]))
@constraint(AL, p[5] >= p[6]+2-(n+1)*(1-x[6,5]))

optimize!(AL)

# collect variables that have value 1 in optimal solution:
vars = []
for i=1:n
    for j=1:n
        if value(x[i,j])>(1.0-1e-5)
            push!(vars,(i,j))
        end
    end
end

#add the new constraint (problem 3):
@constraint(AL, sum(x[i,j] for (i,j) in vars)<=length(vars)-1)

#reoptimize
optimize!(AL)

if termination_status(AL) == MOI.OPTIMAL
    println("\nOptimal Objective Value: $(objective_value(AL))")
    println("\nAircraft Landing times and position\n")
    for i=1:n
        println("Aircraft $i: time = $(round(value(t[i]),digits=2))
        ), position = $(round(value(p[i]),digits=2))")
    end
else
    println("No Solution:")
end
end

```

This assignment does require a little more programming to collect the variables at value one and then define a constraint using these variables. The aim of this assignment is to show you how you can resolve a model using information from the solution to the model.